

Projet Final — Supervised Learning

Prédiction du churn clients (Telco Customer Churn)

Master IA — Module 7 (Capstone) — Enseignant : Ibrahima SY — Année 2025-2026 **Auteure :** Sophie

Mission 0 — Cadrage du problème

Problème métier

Anticiper, parmi les clients actifs, ceux qui présentent un risque élevé de résiliation (churn) dans un futur proche, afin que le service marketing puisse leur adresser en priorité une offre de rétention personnalisée avant qu'ils ne partent.

Utilisateur métier : le service marketing / rétention client, dans le cadre d'une campagne récurrente (mensuelle) de contacts ciblés (appel, e-mail, remise commerciale).

Action déclenchée par la prédiction : un client classé « à risque » entre dans la liste prioritaire de la campagne de rétention ; le score de risque et les facteurs explicatifs (Mission 4, SHAP) permettent au conseiller de choisir une offre adaptée (remise, changement de forfait, geste commercial).

Définition de la cible Churn : variable binaire fournie par l'opérateur, indiquant si le client a résilié son contrat au cours du dernier mois d'observation (Yes/No). C'est un churn *contractuel et déclaré* (résiliation effective), et non une prédiction d'intention — la fenêtre d'observation exacte n'est pas documentée dans le dataset, ce qui est noté comme limite en Mission 5 (model card).

Coûts d'erreur

ARPU (Average Revenue Per User) = revenu moyen mensuel par client, mesuré directement sur la colonne MonthlyCharges du dataset : 64,76 € en moyenne sur l'ensemble des clients, 74,44 € en moyenne pour les clients qui churnent (ils ont tendance à avoir des forfaits plus chers).

	Estimation	Calcul
Faux négatif (churner non détecté, donc non ciblé)	≈ 890 €	ARPU des churners × horizon d'analyse d'un an : 74,44 € × 12 mois ≈ 893 €. On perd le revenu récurrent que ce client aurait généré sur l'année, faute d'avoir pu tenter de le retenir.
Faux positif (offre envoyée à un client fidèle)	≈ 50 €	Remise de rétention –20 % pendant 3 mois sur l'ARPU moyen (0,20 × 65 € × 3 mois = 39 €) + coût de contact (call center / e-mail, ≈ 10 €).

Ratio FN / FP ≈ 18 : 1 (890 / 50) — un churner manqué coûte environ 18 fois plus cher qu'une offre envoyée à tort. Le ratio est calculé coût-le-plus-élevé ÷ coût-le-plus-faible (et non l'inverse) pour se lire simplement : "un FN coûte X fois plus qu'un FP". Le **faux négatif est donc l'erreur la plus grave**, ce qui doit orienter tout le reste du projet (métrique, seuil de décision).

Métrique

- **Métrique principale : le rappel (recall)** de la classe Churn=Yes, complété par le **F2-score** (qui pondère le rappel 4× plus que la précision) pour éviter de dégénérer vers un modèle qui classe tout le monde « à risque ».
- **Pourquoi** : au vu du ratio de coûts ($\approx 18:1$), minimiser les faux négatifs prime largement sur la précision — l'accuracy seule serait trompeuse avec $\sim 27\%$ de churn.
- **Métriques secondaires surveillées** :
 1. **PR-AUC** (plus informative que la ROC-AUC en cas de déséquilibre de classes).
 2. **Précision** de la classe churn, pour éviter de saturer les équipes de rétention avec trop de faux positifs.

Seuil de réussite fixé a priori (avant modélisation) : - Rappel $\geq 80\%$ sur la classe churn - Précision $\geq 45\%$ (au moins ~ 1 offre sur 2 justifiée) - PR-AUC $\geq 0,65$ (à comparer à la baseline naïve $\approx 0,27$, le taux de base du churn)

Risques & hypothèses

- **Stationnarité** : la distribution du churn n'est pas garantie stable dans le temps (nouvelles offres concurrentes, contexte économique, saisonnalité) → nécessite un plan de monitoring post-déploiement (Mission 5).
- **RGPD** : le modèle exploite des données personnelles (démographie, comportement de consommation). Base légale : intérêt légitime pour une action de rétention ; droit d'opposition du client à conserver ; les données de scoring ne doivent pas être conservées au-delà du nécessaire.
- **Latence** : usage en campagne batch (hebdomadaire/mensuelle), pas de contrainte temps réel dur — une latence de l'ordre de la seconde à la minute par lot est acceptable pour l'API.
- **Explicabilité** : nécessaire pour que les conseillers comprennent *pourquoi* un client est à risque et adaptent l'offre en conséquence → SHAP est donc un livrable obligatoire (Mission 4), pas un simple bonus.

Mission 1 — Données et analyse exploratoire

(Détail complet : *notebook notebooks/01_edu.ipynb*.)

Profiling

- **7 043 lignes × 21 colonnes** (20 features + la cible Churn).
- Types : 3 numériques (tenure, MonthlyCharges, TotalCharges), 1 booléen encodé en entier (SeniorCitizen), le reste catégoriel (texte).
- **TotalCharges** est chargée en texte à cause de **11 lignes vides** (chaîne " ") : elles correspondent toutes à des clients avec tenure = 0 (souscription trop récente pour avoir un cumul de charges). Une fois convertie en numérique, cela représente **0,16 % de valeurs manquantes**, uniquement sur cette colonne.
- **Aucun doublon** (ni sur les lignes complètes, ni sur customerID).
- **Aucune feature quasi-constante** (aucune modalité dominante $> 95\%$).
- TotalCharges est fortement corrélée à tenure (0,83) et MonthlyCharges (0,65) — cohérent puisqu'elle en est proche du produit cumulé ; à surveiller pour la colinéarité en modélisation linéaire, mais ce n'est pas une fuite.

Détection de fuite de données

Méthode : AUC en validation croisée (5 plis) de chaque variable prise **isolément** face à Churn. Résultat : aucune feature ne dépasse $\sim 0,70$ d'AUC seule (tenure 0,74, Contract 0,74, puis OnlineSecurity, TechSupport...) — une vraie fuite donnerait un AUC proche de 1,0. Vérification qualitative

complémentaire : le dataset ne contient aucune colonne décrivant un événement *postérieur* au départ du client (pas de Churn Reason/Churn Date). **Conclusion : pas de fuite détectée.** Seul customerID est exclu des features (identifiant sans pouvoir prédictif).

Analyse bivariée et information mutuelle

Top 5 features par information mutuelle avec Churn : **Contract, tenure, OnlineSecurity, Tech-Support, InternetService.**

Hypothèses vérifiées graphiquement

#	Hypothèse	Résultat observé	Verdict
H1	Contrat mensuel → churn plus élevé	42,7 % (mensuel) vs 11,3 % (1 an) vs 2,8 % (2 ans)	Confirmée
H2	Faible ancienneté → churn plus élevé	Médiane 10 mois (churners) vs 38 mois (non-churners)	Confirmée
H3	Fibre optique → churn plus élevé	41,9 % (fibre) vs 19,0 % (DSL) vs 7,4 % (pas d'internet)	Confirmée
H4	Chèque électronique → churn plus élevé	45,3 % vs 15-19 % (autres moyens de paiement)	Confirmée
H5	Pas de support technique → churn plus élevé	41,6 % (sans) vs 15,2 % (avec)	Confirmée

3 insights majeurs

1. **Le type de contrat est le facteur le plus discriminant** (information mutuelle la plus élevée ; écart de churn de 2,8 % à 42,7 %) — les clients sans engagement sont la cible prioritaire de la campagne de rétention.
2. **L'ancienneté est très fortement liée au churn** (médiane 10 vs 38 mois) — les nouveaux clients (< 12 mois) sont une population à risque à surveiller de près.
3. **Le mode de paiement et l'absence de services de protection/support sont des signaux secondaires mais robustes** (chèque électronique 45,3 % de churn ; absence de Tech-Support/OnlineSecurity ~41-42 %), en partie corrélés au type de contrat — à approfondir en feature engineering (Mission 2).

Mission 2 — Préparation et pipeline sans fuite de données

(Détail complet : `notebook notebooks/02_pipeline_baseline.ipynb`.)

Règle respectée : split avant toute transformation apprise

Le `train_test_split` (stratifié, 80/20) est effectué **avant** toute imputation, tout encodage et tout scaling. Répartition obtenue : 5634 clients en train / 1409 en test, avec un taux de churn identique dans les deux (26,5 %), garanti par la stratification.

Imputation de TotalCharges — une règle métier plutôt qu'une médiane aveugle

Plutôt qu'une simple imputation par la médiane, une analyse du mécanisme de la valeur manquante a été menée : les 11 valeurs manquantes de TotalCharges correspondent **exactement** à des clients avec `tenure = 0` (nouveaux clients qui n'ont encore rien payé). Un transformateur scikit-learn personnalisé (TotalChargesImputer) applique donc une règle métier déterministe, appliquée ligne par ligne (donc sans risque de fuite train/test) : - Client nouveau (`tenure = 0`) → TotalCharges = 0 (fait certain, pas une estimation). - Client existant avec valeur manquante (cas non observé ici mais possible en production) → TotalCharges estimé par `tenure × MonthlyCharges`, plus fiable qu'une médiane globale car spécifique à ce client.

Le SimpleImputer(strategy="median") du ColumnTransformer est conservé en filet de sécurité pour toute valeur manquante non couverte par cette règle. Le choix de la **médiane** plutôt que la **moyenne** comme stratégie de repli générique est justifié empiriquement par le skewness des variables numériques du train : `tenure` (0,24) et `MonthlyCharges` (-0,22) sont quasi symétriques (le choix importe peu), mais `TotalCharges` est nettement asymétrique (skewness 0,95 ; moyenne 2302,6 € vs médiane 1398,1 €) — la médiane, robuste aux valeurs extrêmes, est donc le choix le plus sûr des trois.

Pipeline (ColumnTransformer)

- **Numérique** (`tenure`, `MonthlyCharges`, `TotalCharges`) : imputation médiane (filet de sécurité) + StandardScaler.
- **Catégoriel** (16 colonnes, dont `SeniorCitizen` traitée comme catégorielle binaire) : imputation par le mode + OneHotEncoder.

Toutes les statistiques (médiane, mode, catégories) sont apprises uniquement via `.fit()` sur le train, à l'intérieur du pipeline scikit-learn.

Baseline

Régression logistique entraînée dans le pipeline complet, évaluée en validation croisée stratifiée à 5 plis sur le train uniquement :

Métrique	Score (CV)
Rappel	0,544 ± 0,041
F1	0,592 ± 0,030
Accuracy	0,802 ± 0,012

Le rappel (54,4 %) est en dessous du seuil cible de 80 % fixé en Mission 0 — attendu à ce stade : la régression logistique par défaut utilise un seuil de décision de 0,5 sans tenir compte de l'asymétrie des coûts (FN 18× plus cher qu'un FP). L'accuracy (80,2 %) dépasse la baseline naïve (73,5 %, toujours prédire « pas de churn »), mais reste secondaire par rapport au rappel dans notre contexte. **Ce score de rappel (0,544) est la valeur de référence à battre pour la suite du projet** (Mission 3 : autres modèles ; Mission 4 : seuil de décision optimisé).

Feature engineering — testé et rejeté (rasoir d'Occam)

3 features candidates ont été créées à partir des insights de la Mission 1 : `n_services` (nombre de services souscrits), `tenure_bucket` (ancienneté en 3 tranches), `charges_per_tenure` (`TotalCharges / (tenure + 1)`). Chacune a été testée **individuellement** en validation croisée (rappel) :

Configuration	Rappel (CV)	Verdict
Baseline (aucune feature)	0,544 ± 0,041	—
+ <code>n_services</code> seule	0,543 ± 0,041	Neutre

Configuration	Rappel (CV)	Verdict
+ tenure_bucket seule	0,532 ± 0,030	Dégrade
+ charges_per_tenure seule	0,532 ± 0,038	Dégrade

Aucune des 3 features n'apporte de gain réel sur le rappel — tenure_bucket fait perdre de l'information par rapport à tenure continue déjà présente, et charges_per_tenure est trop colinéaire avec MonthlyCharges/ TotalCharges/tenure (corrélations déjà relevées en Mission 1). Les 3 features sont donc **rejetées**, conformément au principe du rasoir d'Occam explicitement demandé par le sujet. Le pipeline retenu pour la suite du projet (baseline_pipeline) reste la version simple : règle métier TotalChargesImputer → ColumnTransformer → modèle.

Mission 3 — Modélisation et comparaison rigoureuse

(Détail complet : notebook notebooks/03_model_comparison.ipynb. Le pipeline de préparation a été factorisé dans src/preprocessing.py, réutilisé telle quelle par cette mission et les suivantes.)

Baseline naïve

Un classifieur qui prédit toujours la classe majoritaire ("pas de churn") obtient 0% de rappel mais 73,5% d'accuracy — illustration directe du piège signalé en Mission 0 : l'accuracy seule seraient trompeuse en cas de déséquilibre de classes. Tout modèle réel doit largement battre ce plancher sur le rappel, notre métrique prioritaire.

3 modèles de familles différentes

- **Régression logistique** (linéaire) — modèle simple et interprétable, déjà notre baseline (Mission 2).
- **Forêt aléatoire** (arbres) — imposée par le sujet, capture des interactions non-linéaires entre variables.
- **SVM à noyau RBF** (famille différente, marges/noyaux) — choix justifié par les caractéristiques des données : après le OneHotEncoder, l'espace de features devient grand et creux (sparse), ce qui pénaliserait un modèle à base de distance comme le k-NN (fléau de la dimension) ; le SVM à noyau reste robuste dans ce contexte. Ses probabilités (probability=True) seront également réutilisées pour la calibration en Mission 4.

Validation croisée honnête (StratifiedKFold, 5 plis, mêmes plis pour tous)

Modèle	Rappel (CV)	Écart-type	Stabilité
Baseline naïve	0,000	—	—
Régression logistique	0,544	±0,041	Moyenne
Forêt aléatoire	0,476	±0,028	Moyenne
SVM (RBF)	0,498	±0,015	La plus stable

Les 3 modèles battent largement la baseline naïve. Résultat notable : le modèle le plus simple (régression logistique) obtient le meilleur rappel, devant la forêt aléatoire (hyperparamètres par défaut, non réglés à ce stade) — illustration concrète du théorème *No Free Lunch* (aucun algorithme n'est universellement meilleur, cf. questions de réflexion). Le SVM, bien que légèrement en retrait sur le rappel moyen, est le modèle le plus stable d'un pli à l'autre (écart-type le plus faible sur les 3 métriques).

Test statistique (Wilcoxon)

Comparaison appariée (mêmes plis) entre les 2 meilleurs modèles sur le rappel — régression logistique vs SVM :

- Différences par pli (LR – SVM) : [0,047 ; 0,017 ; 0,027 ; 0,097 ; 0,040] — **toujours positives**, la régression logistique gagne sur les 5 plis.
- **p-valeur = 0,0625** → **non significatif** au seuil de 0,05.

Limite documentée : avec seulement 5 plis, le test de Wilcoxon exact plafonne mécaniquement à une p-valeur minimale de 0,0625 ($2 \times (1/2^5)$), même si l'écart est parfaitement cohérent dans le même sens sur tous les plis. Une validation croisée à 10 plis donnerait davantage de puissance statistique ; ce choix (5 plis) a été conservé par souci de simplicité, la limite étant explicitement assumée plutôt que dissimulée.

Analyse d'erreurs

Sur les prédictions du meilleur modèle (régression logistique, via `cross_val_predict`) :

- **813 vrais positifs** (churners bien détectés) : **100%** en contrat mensuel, ancienneté médiane de **6 mois** — le modèle excelle sur le profil “évident” (nouveau client, sans engagement) identifié dès la Mission 1.
- **682 faux négatifs** (churners ratés) : **25%** sont en contrat annuel ou biannuel, avec une ancienneté médiane de **25 mois** — un churn “inattendu” chez des clients apparemment fidèles, que le modèle ne capture pas car il s'appuie très fortement sur Contract, la feature la plus discriminante.

Conclusion : aucune feature du dataset actuel ne permet de mieux capturer ce churn tardif/inattendu (cohérent avec le rejet des features de la Mission 2) — limite documentée plutôt qu'un problème résolu à ce stade.

Modèle retenu pour la Mission 4 (tuning, calibration, SHAP) : régression logistique.

Mission 4 — Optimisation, calibration et interprétabilité

(Détail complet : `notebook notebooks/04_tuning_calibration_shap.ipynb`)

Tuning des hyperparamètres (Optuna)

Espace de recherche sur **6 hyperparamètres** de la régression logistique (solveur saga, seul à supporter toutes les régularisations) : C, penalty (l1/l2/elasticnet), l1_ratio, class_weight, max_iter, tol. **60 essais** avec TPESampler + MedianPruner (10 essais coupés en cours de route), objectif = rappel en validation croisée (mêmes plis que la Mission 3).

	Rappel (CV)
Hyperparamètres par défaut	0,544 ± 0,041
Hyperparamètres optimisés	0,803
Gain	+25,9 points

Meilleurs paramètres : C=2,68, penalty='l2', class_weight='balanced', max_iter=2944, tol=0,0006.

Importance des hyperparamètres : class_weight domine très largement (90,5%), loin devant C (6,2%), tol/max_iter (1,6% chacun) et penalty (~0%). Le gain provient presque entièrement du rééquilibrage des classes pendant l'entraînement, qui compense le déséquilibre 73,5%/26,5%.

Calibration des probabilités

Constat avant calibration : le modèle est systématiquement trop confiant (reliability diagram) — ex. pour un groupe de clients à qui il attribue 58% de risque, seuls 32,8% churnent réellement. Cause directe : `class_weight='balanced'` améliore la décision mais déforme les probabilités, qui ne reflètent plus les fréquences réelles.

Après CalibratedClassifierCV (Platt scaling / sigmoid) : l'erreur moyenne de calibration (écart absolu moyen prédit/observé sur 10 groupes) passe de **0,148 à 0,012** — amélioration d'un facteur 12. Une probabilité de churn fiable est indispensable pour prioriser finement les offres de rétention (ex. remise plus importante pour un client à 80% de risque qu'à 55%) plutôt que de traiter tous les clients "à risque" de façon uniforme.

Interprétabilité (SHAP)

Importance globale (top 5) : MonthlyCharges, tenure, InternetService_Fiber optic, TotalCharges, InternetService_DSL. Ordre différent de l'information mutuelle de la Mission 1 (où Contract dominait) : SHAP reflète les poids du modèle final réglé, alors que l'information mutuelle mesurait un lien brut indépendant du modèle ; Contract reste très présent mais son importance est répartie entre ses modalités one-hot.

Summary plot : confirme les tendances métier attendues — faible ancienneté et charges élevées poussent vers le churn (points colorés cohérents avec les insights de la Mission 1).

3 décisions individuelles (waterfall plots) : un vrai positif, un vrai négatif et un faux positif ont été expliqués individuellement, montrant comment chaque variable contribue à la probabilité finale pour un client donné — utile pour qu'un conseiller comprenne le "pourquoi" du score avant de personnaliser une offre.

Dependence plot (MonthlyCharges, feature dominante) : relation strictement linéaire et **décroissante** (contre-intuitive au premier abord, puisque les churners ont en moyenne des charges plus élevées en Mission 1). Explication : forte colinéarité entre MonthlyCharges, tenure et TotalCharges (corrélations 0,65-0,83, Mission 1) — une fois tenure et TotalCharges déjà pris en compte par le modèle, l'effet marginal résiduel de MonthlyCharges peut changer de signe. **Limite d'interprétabilité documentée :** les poids d'un modèle linéaire avec variables corrélées ne reflètent pas nécessairement les corrélations brutes univariées.

Choix du seuil de décision

Recherche du seuil minimisant le coût métier total ($890\text{€} \times \text{FN} + 50\text{€} \times \text{FP}$, Mission 0) sur les probabilités calibrées :

Seuil	Rappel (test)	Précision (test)	Coût total (test)
0,5 (défaut)	0,559	—	152 200 €
0,06 (optimisé, retenu)	0,979	0,366	38 770 €

Réduction de 74,5% du coût métier total sur le jeu de test (jamais vu avant cette étape), en changeant uniquement le seuil de décision. Le seuil optimal (0,06) est très inférieur à 0,5, confirmant l'intuition de la Mission 0 : au vu du ratio de coûts (18:1), le modèle doit être beaucoup plus "alerte" — mieux vaut multiplier les fausses alertes (50€ chacune) que rater des churners (890€ chacun).

Compromis assumé : la précision (36,6%) est en dessous du seuil secondaire de 45% fixé en Mission 0. C'est économiquement rationnel vu le ratio de coûts, mais représente une charge de travail conséquente pour les équipes de rétention (633 fausses alertes sur 1409 clients test) — point à surveiller en production et à mentionner comme limite du seuil purement cost-minimisant (une contrainte de capacité opérationnelle pourrait justifier un seuil légèrement plus prudent).

Résultat final retenu pour la Mission 5 : pipeline = TotalChargesImputer → ColumnTransformer → régression logistique optimisée (Optuna) → CalibratedClassifierCV (Platt) → seuil de décision = 0,06.

Mission 5 — Déploiement, tests et monitoring

(Détail complet: `src/train_final_model.py`, `tests/test_pipeline.py`, `api/main.py`, `README.md`, `MODEL_CARD.md`.)

Sérialisation

Le pipeline final (hyperparamètres Optuna figés, calibration Platt, réentraîné sur 100% des données disponibles après évaluation sur le test) est exporté avec `joblib` dans `model/pipeline.joblib`, accompagné de `model/model_metadata.json` (version, seuil de décision, features attendues, performance de validation). Un test dédié (`test_reload_gives_identical_predictions`) confirme que deux rechargements successifs du fichier produisent des probabilités strictement identiques (`np.array_equal`).

Tests (pytest)

6 tests (le sujet en demande au moins 5), tous passants :

Test	Vérifie
<code>test_output_shape</code>	La sortie a la bonne forme (une prédiction par ligne)
<code>test_probabilities_in_valid_range</code>	Les probabilités sont dans [0, 1] et somment à 1
<code>test_handles_missing_values</code>	Le pipeline gère les valeurs manquantes sans planter
<code>test_expected_features_present</code>	Les features attendues sont présentes
<code>test_performance_on_reference_set</code>	La performance sur un mini-jeu de référence reste correcte
<code>test_reload_gives_identical_predictions</code>	Rechargement = prédictions strictement identiques

6 passed in 22.63s

API REST (FastAPI)

Trois endpoints exposés et testés avec `curl` :

- **GET /health** → `{"status": "ok", "version": "1.0.0"}`
- **GET /model-info** → features attendues, seuil de décision, performance de validation.
- **POST /predict** → exemple testé (client fibre optique, contrat mensuel, faible ancienneté, charges élevées): `{"churn_prediction": 1, "churn_probability": 0.6004, "decision_threshold": 0.0}`
— cohérent avec le profil à risque identifié dès la Mission 1.

Performance par sous-groupe (fairness, complète la model card)

Sous-groupe	Rappel (test)	n
Genre — Homme	0,989	722
Genre — Femme	0,969	687

Sous-groupe	Rappel (test)	n
Senior	0,990	222
Non-senior	0,975	1187

Écart maximal entre sous-groupes : 2,1 points — performance homogène sur les deux critères démographiques disponibles dans le dataset.

Documentation

- **README.md** : problème métier, données, modèle, installation, entraînement, tests, lancement et usage de l'API.
- **MODEL_CARD.md** : fiche standardisée (inspirée de Mitchell et al., FAccT 2019) — détails du modèle, usage prévu et hors périmètre, données d'entraînement, performance globale et par sous-groupe, limites connues (précision faible assumée, churn tardif mal détecté, colinéarité SHAP, fenêtre de churn non documentée, stationnarité non garantie, fairness partiellement évaluée).

Pourquoi une model card : c'est un standard qui documente un modèle comme une fiche technique — utile pour l'auditabilité, la confiance et la responsabilité, en particulier pour des modèles influençant des décisions métier (ici, qui reçoit une offre de rétention).

Plan de monitoring

Trois signaux à surveiller en production :

1. **Data drift** — la distribution des features change-t-elle par rapport à l'entraînement (ex. hausse générale de MonthlyCharges) ? Détection : test de Kolmogorov-Smirnov ou PSI (Population Stability Index) par variable.
2. **Concept drift** — la relation features→churn change-t-elle (ex. un contexte économique qui modifie le comportement des clients) ? Détection : nécessite le vrai résultat, donc avec un décalage temporel.
3. **Performance drift** — le rappel/la précision réels en production se dégradent-ils par rapport à la Mission 4 (rappel 0,979) ?

Outil recommandé : **Evidently** (open-source, Python) — compare un jeu de référence (train) à des lots de production, génère des rapports de dérive par variable. **Cadence proposée** : rapport hebdomadaire/mensuel ; réentraînement déclenché si $PSI > 0,2$ sur une feature clé (Contract, MonthlyCharges) ou si le rappel réel mesuré descend sous le seuil cible de 80% (Mission 0).

Questions de réflexion

1. Concept drift

Un modèle bon à l'entraînement apprend des régularités statistiques présentes dans les données historiques (ex. "contrat mensuel → churn élevé"). En production, ces régularités peuvent cesser d'être valides sans que rien ne change dans le code du modèle : une nouvelle offre concurrente, une évolution macroéconomique, ou un changement de politique tarifaire peuvent modifier la **relation** entre les features et le churn (le "concept" que le modèle a appris), même si les features elles-mêmes restent dans les mêmes plages de valeurs. Le modèle continue alors à produire des scores avec la même confiance apparente, mais ces scores deviennent de moins en moins fiables — c'est un échec silencieux, plus dangereux qu'une erreur visible. **Ce qu'il faut surveiller** (détaillé en Mission 5) : la performance réelle une fois le résultat connu (rappel/précision en production comparés à la Mission 4), et en amont, la stabilité de la distribution des features clés (Contract, tenure, MonthlyCharges) pour détecter un signal avant-coureur.

2. Information mutuelle vs corrélation de Pearson

La corrélation de Pearson ne mesure que les dépendances **linéaires** entre deux variables numériques : deux variables peuvent être fortement liées de façon non-linéaire (ex. une relation en U, ou un effet de seuil) sans que Pearson ne détecte quoi que ce soit (corrélation proche de 0 malgré une vraie dépendance). L'information mutuelle, elle, mesure la réduction d'incertitude sur une variable en connaissant l'autre, **quelle que soit la forme** de la relation (linéaire, non-linéaire, non-monotone), et s'applique nativement aux variables catégorielles, sans les supposer numériques ou ordonnées. C'est précisément pour ça qu'on l'a choisie en Mission 1 : elle a permis de classer sur un pied d'égalité des variables numériques (tenure) et catégorielles (Contract, PaymentMethod) dans un seul classement, ce que Pearson n'aurait pas permis directement sans encodage préalable ni hypothèse de linéarité.

3. No Free Lunch

Le théorème du *No Free Lunch* établit que, moyenné sur l'ensemble de tous les problèmes possibles, aucun algorithme d'apprentissage n'est meilleur qu'un autre — la performance d'un modèle dépend de l'adéquation entre son biais inductif (les hypothèses qu'il fait implicitement sur la structure des données) et la structure réelle du problème traité, qui varie d'un dataset à l'autre. Concrètement, on ne peut pas savoir à l'avance quel modèle conviendra le mieux à *nos* données sans le tester empiriquement. C'est exactement ce que la Mission 3 a illustré : contre l'intuition qu'un modèle plus sophistiqué (forêt aléatoire) devrait battre un modèle simple, c'est la **régression logistique** qui a obtenu le meilleur rappel avec des hyperparamètres par défaut. Sans le benchmark comparatif rigoureux exigé par le sujet, ce résultat contre-intuitif serait passé inaperçu, et un modèle sous-optimal aurait pu être choisi par simple présomption de supériorité.

4. Fairness

Deux métriques classiques pour évaluer l'équité d'un modèle entre sous-groupes :

- **Demographic parity (parité démographique)** : la proportion de clients classés "à risque" doit être similaire entre sous-groupes, indépendamment du résultat réel — $P(\hat{y}=1 \mid \text{groupe A}) \approx P(\hat{y}=1 \mid \text{groupe B})$. Se mesure en comparant le taux de prédictions positives par groupe (un ratio proche de 1, ou la règle empirique des 80% utilisée en pratique, indique une bonne parité).
- **Equalized odds (chances égalisées)** : le modèle doit avoir le même taux de vrais positifs (rappel) **et** le même taux de faux positifs entre sous-groupes, conditionnellement au résultat réel — $P(\hat{y}=1 \mid y=1, \text{groupe A}) \approx P(\hat{y}=1 \mid y=1, \text{groupe B})$ et la même égalité sur les faux positifs. Se mesure en calculant une matrice de confusion séparée par sous-groupe et en comparant les taux.

Notre modèle est-il équitable ? La Mission 5 a mesuré le rappel par sous-groupe (genre, senior) : écart maximal de 2,1 points (0,969 à 0,990) — une bonne indication sur le rappel (proche de l'equalized odds côté vrais positifs), mais **incomplète** : le taux de faux positifs par sous-groupe et la demographic parity n'ont pas été calculés, et seules deux variables démographiques étaient disponibles dans le dataset. C'est une limite documentée plutôt qu'une garantie d'équité totale.

5. Fuite de données — inventaire des points de vigilance

Récapitulatif des endroits précis du pipeline où une fuite aurait pu s'introduire, et comment elle a été évitée à chaque étape :

Endroit à risque	Fuite possible	Comment elle a été évitée
Split train/test	Faire le split après une transformation apprise	Split stratifié effectué avant toute transformation (Mission 2), premier réflexe du projet

Endroit à risque	Fuite possible	Comment elle a été évitée
Imputation (médiane/mode)	Calculer la médiane/mode sur tout le dataset	SimpleImputer à l'intérieur du Pipeline, fit uniquement sur le train (ou le pli d'entraînement en CV)
Scaling (StandardScaler)	Apprendre moyenne/écart-type sur test inclus	Idem — appris uniquement sur le train via le pipeline
One-Hot Encoding	Catégories vues uniquement sur le test	handle_unknown="ignore", catégories apprises sur le train uniquement
Règle métier TotalChargesImputer	—	Aucune statistique apprise du dataset : transformation ligne par ligne (tenure x MonthlyCharges de la même ligne), donc structurellement sans risque
Feature engineering (Mission 2)	Tester le gain sur tout le dataset avant de trancher	Évalué par validation croisée avec le transformateur dans le pipeline (recalculé à chaque pli)
Tuning Optuna (Mission 4)	Choisir les hyperparamètres en regardant le test	Objectif optimisé uniquement sur le train (CV), test jamais consulté pendant les 60 essais
Calibration (Mission 4)	Calibrer avec des données du test	CalibratedClassifierCV avec cross-fitting sur le train uniquement
Seuil de décision (Mission 4)	Choisir le seuil en regardant le test	Seuil optimisé sur les probabilités out-of-fold du train ; le test n'a servi qu'à la confirmation finale, une fois le seuil déjà figé
Fuite target (Mission 1)	Une feature encode le résultat (ex. raison de résiliation)	Vérifiée par AUC de chaque variable seule (aucune ne dépasse ~0,74) + inspection qualitative des colonnes
Refit final (Mission 5)	Le refit sur 100% des données fausse-t-il les métriques rapportées ?	Non : le refit sur train+test a lieu après que toutes les métriques de performance ont été calculées et figées sur le split initial — il ne sert qu'au modèle déployé, pas à l'évaluation

Aucune fuite de données identifiée — chaque étape a été conçue, documentée et vérifiée pour respecter la règle fondamentale du projet : toute transformation apprise vit dans un pipeline fit sur le train seul.